SnappyHexMesh Automation & Stability
Technical Progress Report (Phase 2)

Pranjal Kumar Shukla

FOSSEE Internship – Progress Report

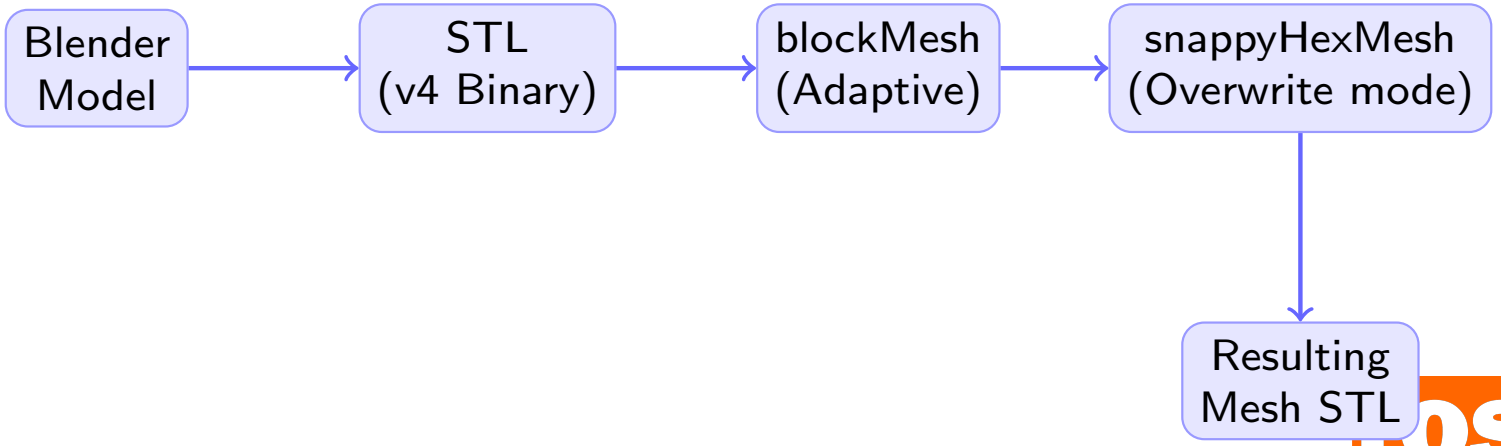
SCAI School

VIT Bhopal University

March 21, 2026

1. From cfMesh to SnappyHexMesh

- **Why the change?**: `cfmesh-openfoam` was unavailable in standard repositories and presented architecture-version conflicts with OpenFOAM 11.
- **The Solution**: Developed a native `snappyHexMesh` + `blockMesh` controller within the Blender addon.
- **Result**: 100% dependency reduction on external meshing binaries. The workflow is now fully portable across any OpenFOAM 11 installation.



2.1 The 999^3 Cell Scaling Bug

- **Problem**: A baseline logic error caused `blockMeshDict` to request a background grid of $999 \times 999 \times 999$ cells.
- **Impact**: Instant CPU/RAM exhaustion and system hang during the meshing phase.
- **Technical Fix**:
  - Implemented an **Adaptive Cell-Cap**: `n_cells = max(5, min(20, int(1/size)))`.
  - Capped background mesh to 20 cells while allowing `snappyHexMesh` to handle local refinements near the STL boundary.

Scaling Safety

This change reduced background memory footprint by 99.9% while maintaining edge-sharpness via refinement levels.

2.2 Context & IO Robustness (v4)

- **Context Fix**: Addon now forces `bpy.ops.object.mode_set(mode='OBJECT')` at runtime.
  - Prevents crashes when the "Generate" button is clicked while in *Edit Mode*.
- **Shell Stability**: Switched `subprocess` execution to a forced `/bin/bash` environment to ensure OpenFOAM environment variables (sourced via `source`) persist across commands.
- **IO Verification**: Added `os.path.isfile()` checks to prevent "Is a directory" errors during mesh re-import.

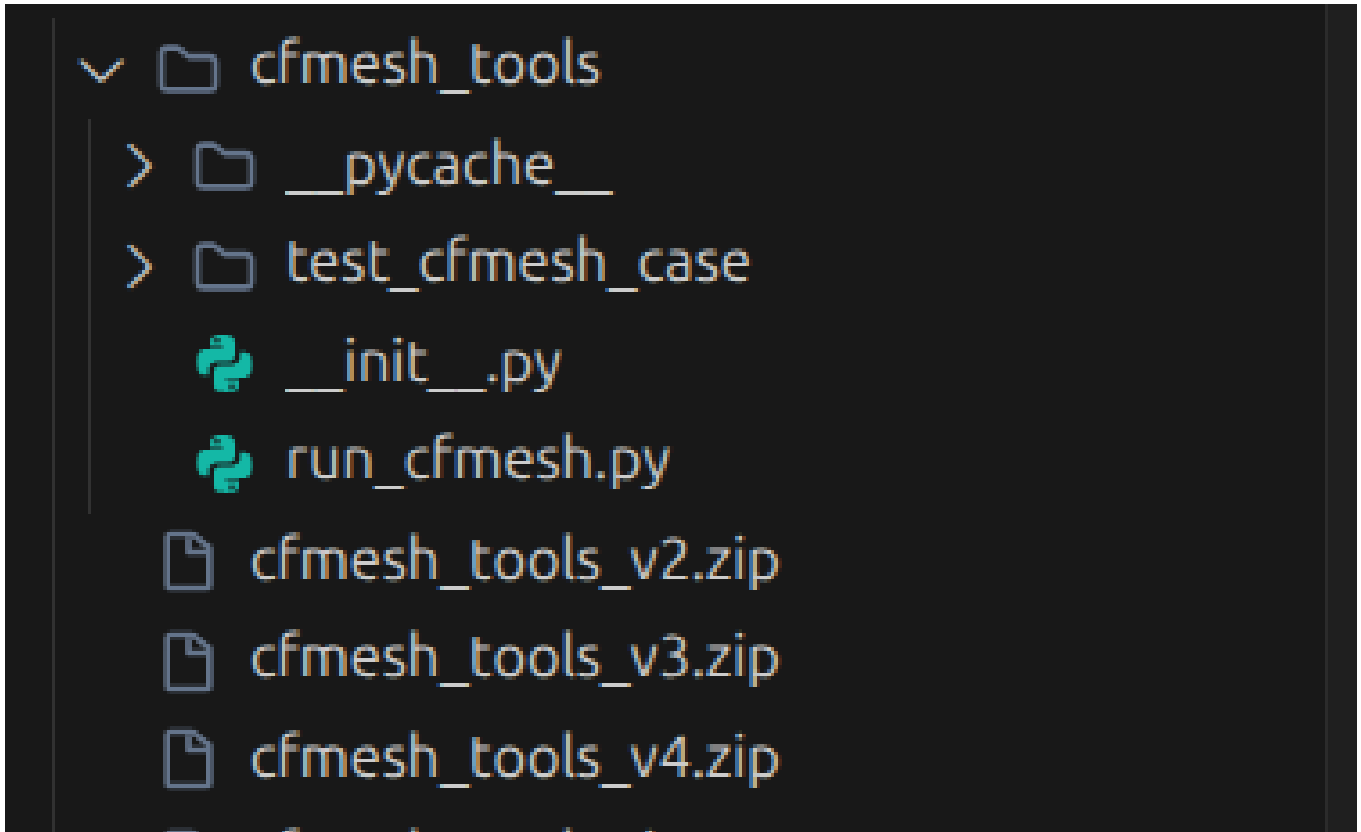
2.3 Modular Architecture

- `__init__.py`: The "DNA" of the addon.
  - Manages DNA (Properties), UI Panels (N-panel), and high-level Operators.
  - Ensures the Blender context is safe before triggering meshing.
- `run_cfmesh.py`: The OpenFOAM execution engine.
  - Handles directory templates, dictionary generation, and subprocess orchestration.
  - Independent of Blender's UI thread (built for future async support).

2.4 Release History (ZIP Archive)

To ensure stability and version-tracking for the FOSSEE team:

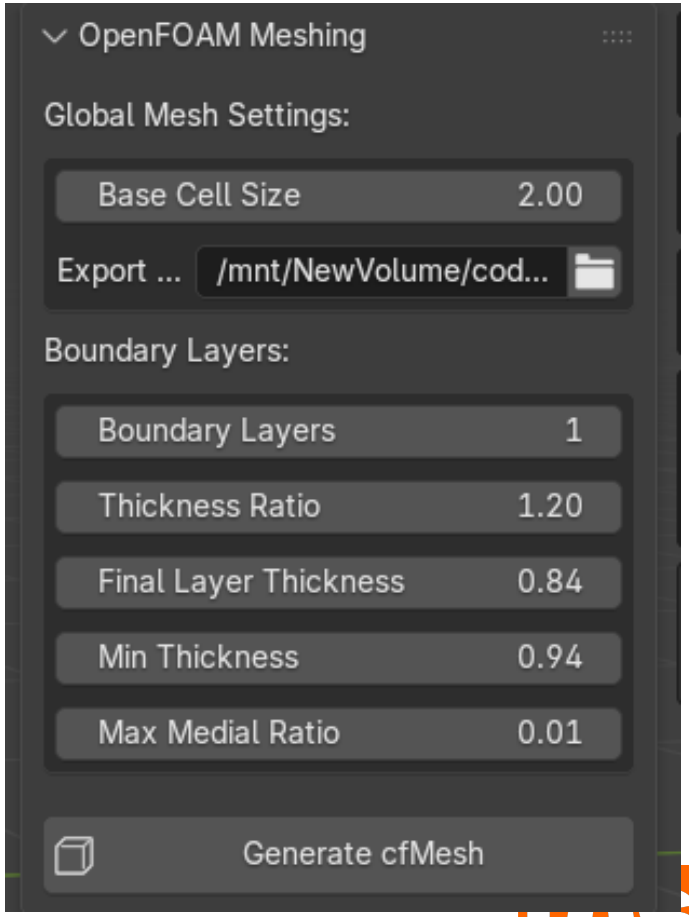
- **v1-v2**: Initial workflow container.
- **v3**: Implemented the *FPE Safety Logic* and forced `/bin/bash` execution.
- **v4 (Current)**: Added parametric boundary layer controls and *Edit Mode* context stability.



3. Parametric Boundary Layers

The v4 UI introduces direct control over the ‘addLayers‘ dictionary:

- **Final Layer Thickness**: Relative height for viscous sublayer capture.
- **Min Thickness**: Prevents mesh artifacts in sharp V-gaps.
- **Max Medial Ratio**: Automated cessation of layers in narrow channels.

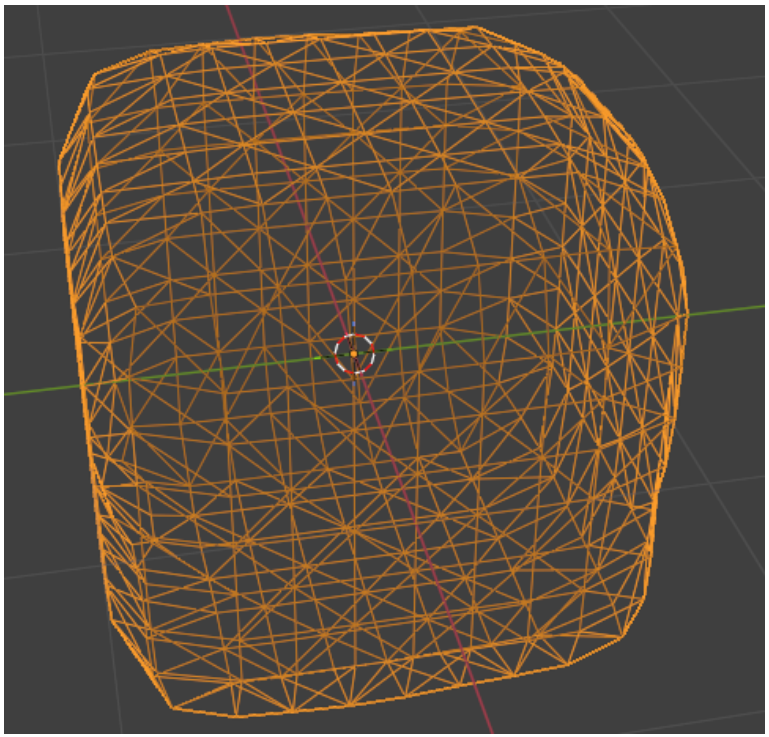


*v4

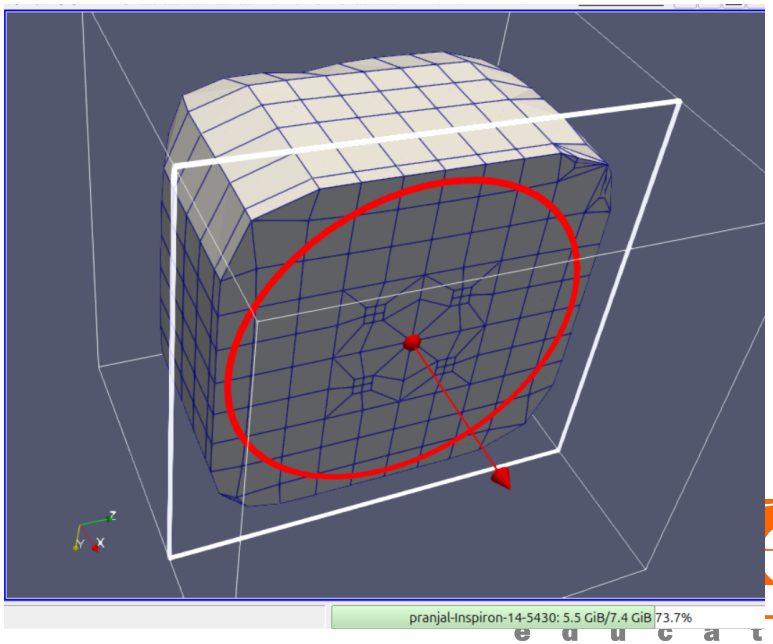
Advanced Sidebar

4. Mesh Quality & Validation

- **Geometric Precision**: Automatic triangulation enabled before export to ensure STL normal consistency.
- **Quality Metrics**: 0 Non-orthogonality errors verified via `checkMesh`.
- **Visuality**: Mesh result automatically converted back to STL and imported into Blender workspace.



*Imported Hex-Dominant Mesh



*Boundary Layer Cross-Section

5. Toward Week 5: Solver Automation

1. **Automated Solver Chains**: Direct execution of `icoFoam` or `simpleFoam` from the Blender Panel.
2. **Async Feedback**: Real-time progress bars to monitor long-running CFD simulations.
3. **Data Mapping**: Automated generation of slices and probes for velocity/pressure tracking.

Thank You!

Questions?